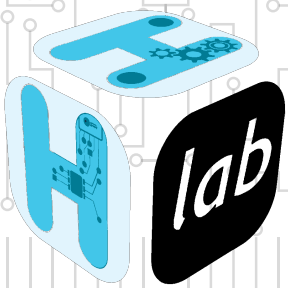
H2LAB-STIG-MAJ
07-2026

Embedded Systems: Guide to Secure Updates



STIG H²Lab

Information

Created in 2020, H2Lab is a non-profit organization dedicated to designing, developing, and sharing open-hardware and open-source solutions for experimentation around embedded systems. Its main areas of work are:

- Design of hardware platforms for hardware and software analysis
- Development of open alternatives to proprietary, often opaque tools
- Publication of technical guides and recommendations to promote bestpractices in security, privacy, and sustainability across the ecosystemof connected devices and embedded systems

Support for researchers, teachers, and students through the provision of tools, educational content, and training.
Academic partnerships to encourage sharing of resources and knowledge.

Releases history

Version	**Date**	**Auteur**	**Modifications**
1.0	July 2026	H2Lab	Initioal guide release

Contents

1
Glossary and acronyms

1.1 Glossary

Trust anchor. Immutable data (public key or hash) used to establish the verification chain for boot and updates.

Anti-rollback. Mechanism preventing the installation of an earlier, potentially vulnerable version, even when it is correctly signed.

Active bank. Firmware slot currently executed at boot.

Inactive bank. Firmware slot targeted by the next update.

Boot vector / vector table. Table mapping interrupts/exceptions to their handlers; its integrity determines control over the execution flow.

Fail-closed. Default security behavior on error: reject rather than accept.

GoT (Global Offset Table). Table of pointers/offsets resolved at load time, allowing global references to be redirected without rewriting all the code.

Boot state machine. Automaton driving the *pending/confirmed/rollback/recovery* transitions.

Update manifest. Signed metadata describing version, compatibility, installation constraints, fingerprints and associated policies.

PI-lite. Pragmatic variant of position-independent code, limiting relocation to critical blocks to contain the RAM/Flash/performance cost.

Power-fail safe. Property of a mechanism that remains resilient to power loss in the middle of a critical operation.

Secure boot. Cryptographic verification of the integrity and authenticity of software stages before execution.

TOCTOU. *Time Of Check To Time Of Use*: inconsistency between the object that was verified and the object actually used.

Wear-leveling. Distribution of writes across the NVM to reduce local wear and extend memory lifetime.

1.2 List of acronyms

AES-CTR. *Advanced Encryption Standard* in counter mode.

AES-GCM. *Advanced Encryption Standard* in Galois/Counter mode.

AM. Attacker profile formalized in this guide (AM-01 to AM-05).

ANSSI. French national agency for information systems security.

APDU. *Application Protocol Data Unit*.

API. *Application Programming Interface*.

APT. *Advanced Persistent Threat*.

BL2. *Boot Loader stage 2* (secondary boot stage).

BLE. *Bluetooth Low Energy*.

CA. *Certificate Authority*.

CAP. Technical capability associated with an attacker profile (CAP-01 to CAP-08).

CBOR. *Concise Binary Object Representation*.

CI/CD. *Continuous Integration / Continuous Delivery*.

COSE. *CBOR Object Signing and Encryption*.

CRA. *Cyber Resilience Act* (Regulation (EU) 2024/2847).

CRC. *Cyclic Redundancy Check*.

CRL. *Certificate Revocation List*.

CVE. *Common Vulnerabilities and Exposures*.

DEK. *Data Encryption Key*.

DICE. *Device Identifier Composition Engine*.

DFU. *Device Firmware Upgrade*.

ECC. *Error Correcting Code*.

ECDSA. *Elliptic Curve Digital Signature Algorithm*.

EM. *Electromagnetic*.

EMFI. *ElectroMagnetic Fault Injection*.

ENC. *Encryption* (encryption key usage).

ETSI. *European Telecommunications Standards Institute*.

EU. *European Union*.

FIPS. *Federal Information Processing Standards.*

FS. Security function required in this guide (FS-01 to FS-09).

GOT. *Global Offset Table.*

HKDF. *HMAC-based Key Derivation Function.*

HMAC. *Hash-based Message Authentication Code.*

HTTP. *HyperText Transfer Protocol.*

HW. *Hardware.*

HSM. *Hardware Security Module.*

ICS. *Industrial Control System.*

ID. Identifier.

IEC. *International Electrotechnical Commission.*

IP. *Internet Protocol.*

IV. *Initialization Vector.*

JTAG. *Joint Test Action Group* (debug interface).

KDF. *Key Derivation Function.*

KMS. *Key Management Service.*

MAC. *Message Authentication Code.*

MCU. *Microcontroller Unit.*

MFA. *Multi-Factor Authentication.*

MITM. *Man-In-The-Middle.*

ML-DSA. *Module-Lattice Digital Signature Algorithm.*

ML-KEM. *Module-Lattice Key Encapsulation Mechanism.*

MMIO. *Memory-Mapped Input/Output.*

MMU. *Memory Management Unit.*

MPU. *Memory Protection Unit.*

MQTT. *Message Queuing Telemetry Transport.*

NA4. *Naturally Aligned 4-byte region* (RISC-V PMP mode).

NAPOT. *Naturally Aligned Power-Of-Two* (RISC-V PMP mode).

NIST. *National Institute of Standards and Technology.*

NISTIR. *NIST Interagency/Internal Report.*

NOR. NOR Flash memory family.

NVM. *Non-Volatile Memory.*

OCSP. *Online Certificate Status Protocol.*

OS. Security objective in this guide (OS-01 to OS-08).

OTA. *Over-The-Air* (remote update).

OTP. *One-Time Programmable.*

PC. *Program Counter.*

PCB. *Printed Circuit Board.*

PCROP. *Proprietary Code Read-Out Protection.*

PI. *Position-Independent.*

PIC. *Position-Independent Code.*

PIE. *Position-Independent Executable.*

PKI. *Public Key Infrastructure.*

PMP. *Physical Memory Protection* (RISC-V).

PQC. *Post-Quantum Cryptography.*

PSA. *Platform Security Architecture.*

RAM. *Random Access Memory.*

RCE. *Remote Code Execution.*

RDP. *Readout Protection.*

RDP2. Strictest STM32 readout protection level (level 2).

RIoT. *Robust Internet of Things* identity architecture.

RISC-V. *Reduced Instruction Set Computer V.*

RMA. *Return Merchandise Authorization* (maintenance/factory return).

ROM. *Read-Only Memory.*

RoT. *Root of Trust.*

RV32. 32-bit RISC-V family.

SAU. *Security Attribution Unit* (TrustZone-M).

SBOM. *Software Bill of Materials.*

SCADA. *Supervisory Control And Data Acquisition.*

SCP03. *Secure Channel Protocol 03* (GlobalPlatform).

SE. *Secure Element.*

SHA-256. *Secure Hash Algorithm 256-bit.*

SIS. *Safety Instrumented System.*

SKU. *Stock Keeping Unit.*

SOC. *Security Operations Center.*

SP. *Special Publication* (NIST SP).

SUIT. *Software Updates for Internet of Things.*

SWD. *Serial Wire Debug.*

TF-M. *Trusted Firmware-M.*

TLS. *Transport Layer Security.*

TOCTOU. *Time Of Check To Time Of Use.*

TOR. *Top Of Range* (RISC-V PMP mode).

TUF. *The Update Framework.*

UART. *Universal Asynchronous Receiver-Transmitter.*

UICC. *Universal Integrated Circuit Card.*

USB. *Universal Serial Bus.*

USB-IF. *USB Implementers Forum.*

VTOR. *Vector Table Offset Register* (Cortex-M).

XIP. *eXecute In Place.*

2
Introduction and scope

2.1 Purpose of the guide

This document provides a reference architecture for the secure update of microcontrollers in constrained contexts: little RAM, internal Flash with coarse erase granularity, no full memory abstraction (no MMU, sometimes a minimal MPU), and strong service continuity requirements. The goal is to make the update architecture:

- resistant to modern attacks on the update chain;
- operationally robust (power loss, unexpected reset, partial image);
- compatible with platforms with or without dual-bank NVM support.

2.2 Target audience

The guide targets embedded architects, product security teams, firmware platform owners, PKI/DevSecOps teams and compliance assessors (IEC 62443, ETSI EN 303 645, NISTIR 8259A, PSA Certified).

2.3 Technical scope and assumptions

2.3.1 Target equipment type

The scope mainly covers:

- IoT- and industrial-class Cortex-M/RISC-V MCUs;
- firmware stored in internal Flash or in external NOR executed in place (XIP);
- initial boot from the manufacturer ROM, followed by a secondary boot chain such as MCUboot, TF-M BL2 or equivalent [21, 46, 6].

2.3.2 Covered architectures: Cortex-M and RV32

The document explicitly covers two target families:

- **Arm Cortex-M**: boot vector management through the vector table and VTOR, memory-mapped protection with the MPU and, depending on the SKU, additional compartmentalization through TrustZone-M [18].
- **RISC-V RV32**: `M/S/U` privilege model, exception routing through `mtvec`, and memory access control through the **PMP** (TOR, NAPOT and NA4 modes) configured by the secure boot in M-mode [40, 18].

On RV32, the practical equivalent of a "secure boot + software anti-tamper" policy rests on a triptych: an immutable anchor, a locked PMP policy (`L` bit) and strict isolation of the update metadata.

2.3.3 Constraints of an MCU without memory abstraction

On an MCU without advanced memory abstraction:

- code is often linked to fixed addresses (interrupt vectors, absolute sections);
- dynamic relocation is limited, if available at all;
- the protection of critical areas relies on hardware security options (RDP, PCROP, secure boot fuses, TrustZone-M) rather than on virtual isolation.

This calls for significant work on the linker script, the vector table (VTOR), and the inter-bank call constraints when aiming for behavior close to position-independent code.

2.3.4 Essential technical background

Wear-leveling. Update lifecycle markers must never be rewritten in place on the same Flash cell. Journal into a ring buffer, use monotonic sequence numbers, and validate every record with CRC/ECC.

PIC / PI-lite. A full PIC approach remains expensive on constrained MCUs. The usual industrial compromise is to make only the critical sections relocatable (boot trampoline, interrupt stubs, service pointers) and to limit absolute references.

Boot vector. On Cortex-M, vector relocation is centralized through VTOR. On RV32, the equivalent is achieved through `mtvec`, distinguishing direct from vectored mode; the bootloader→application transition must guarantee consistent exception handlers.

2.3.5 Operation and maintenance assumptions

The system is assumed to operate 24/7, with periodic fleet-wide updates. Threats include a remote attacker, a local attacker with debug access, and a physical attacker with fault injection capabilities [38, 26].

3

Cyber context and rationale for the A/B architecture

3.1 Targeted threats

Firmware tampering. The basic attack consists of injecting an unauthorized binary or modifying a legitimate artifact in transit. Lessons learned from securing software repositories (TUF/Uptane) show that a signature alone is not enough without consistent metadata, expiration, role delegation and anti-replay [44, 47].

Downgrade and update replay. An old but correctly signed firmware can reintroduce a fixed CVE. Mitigation relies on a monotonic counter in hardware (OTP/eFuse) or a minimum-version policy enforceable in the *boot verifier* [24, 25].

Distribution chain compromise. Supply-chain incidents (e.g. SolarWinds) require isolating the signing roles, logging every issuance and keeping revocation mechanisms operable in degraded mode [31, 9].

Lessons from known attacks. The following cases structure the defender's state of the art. Beyond the incident's name, security engineering must spell out the chronology, the type of attacker, the targeted asset and the expected operational gain. This reading eases the transition from "lessons learned" to testable requirements on the update chain.

Denial-of-service risk during an update. A DoS can be intentional (repeatedly invalid image, oversized payload) or accidental (power cut). A/B is chosen primarily to turn an update failure into a controlled rollback rather than an irreversible outage.

3.2 Attacker model and technical capabilities

The threat model is formalized along two dimensions:

- **AM-* profiles**: the attacker's position in the system (remote, local, physical).
- **CAP-* capabilities**: the technical means actually at the attacker's disposal.

Case (date)	Attacker	Attack	Target	Associated gain
Stuxnet (2010)	Highly capable state actor (converging public attribution)	Software chain and industrial controllers	ICS/SCADA environment and physical processes	Covert sabotage, persistence, kinetic effect through malicious code signed or perceived as legitimate [15]
Jeep Uconnect (2015)	Offensive researchers, scenario transposable to an opportunistic remote attacker	Remote infotainment/telematics surface, network pivot to internal buses	Connected vehicle and critical functions	Remote takeover of vehicle functions, demonstration of security/safety impact [23]
TRITON/TRISIS (2017)	Advanced group targeting OT (publicly attributed)	Compromise of engineering workstations, then of safety components	SIS/industrial safety controllers	Neutralization of safety barriers, increased potential for a major incident [10]
SolarWinds (2020)	APT actor on the software supply chain	Corruption of the integration/publication chain and compromised software distribution	Vendor, integrators, end customers	Mass distribution of backdoors through a legitimate update mechanism [9]
Fault injection on MCUs (2017–2024)	Local/physical attacker with lab equipment	Voltage/clock glitching, EMFI, debug/-boot control bypass	Microcontrollers in the attacker's possession	Secret extraction, secure boot/readout protection bypass, preparation of large-scale attacks [37, 5, 26]

Table 3.1: Background of the reference attacks

3.3 Security objectives associated with the dual bank

The security objectives are normalized below under an `OS-*` naming scheme.

Required security functions (FS-*). The implementable security functions are identified under an `FS-*` naming scheme and linked to the `OS-*` objectives.

Threat → objectives → functions traceability. The matrix below makes the security justification chain explicit.

Continuity of service and security. A known-valid bank must remain available as long as the candidate is not confirmed. This rule prevents a transient failure from turning an update campaign into massive unavailability, and it imposes explicit management of boot states and retry thresholds.

Software integrity before execution. The jump to the candidate bank must only occur after a complete cryptographic check: signature, target compatibility, validity window and anti-rollback policy. This verification must be deny-by-default and auditable, to reduce acceptance ambiguities.

Recoverability on update failure. The design must tolerate power loss: any interruption between the write, the *pending* marking and the switchover must converge to a deterministic, recoverable state. Transition metadata and retry counters must therefore be redundant and consistent.

Traceability and auditability of changes. Every boot state transition (candidate received,

ID	Profile	Main surface	Capability assumptions
AM-01	Opportunistic remote	OTA API, IP network, edge backend	CAP-01, CAP-02, CAP-03
AM-02	Advanced remote (APT)	Software supply chain, CI/CD, operational PKI	CAP-01 to CAP-05
AM-03	Malicious local maintainer	Debug port, maintenance interfaces, UART/SWD/JTAG	CAP-02, CAP-04, CAP-06
AM-04	Physical, in a laboratory	PCB, power supply, clock, EM, memory extraction	CAP-06, CAP-07, CAP-08
AM-05	Ecosystem insider	Publication tools, artifact repository, key policies	CAP-03, CAP-04, CAP-05

Table 3.2: Attacker profiles considered

ID	Capability	Operational description
CAP-01	Network injection/replay	OTA payload injection, replay of old artifacts, partial MITM.
CAP-02	Software exploitation	Application RCE, software privilege escalation, TOCTOU on the update agent.
CAP-03	Supply-chain compromise	Artifact substitution, corruption of the integration/signing chain, publication of compromised packages.
CAP-04	Theft/abuse of secrets	Illegitimate use of publication keys, CI tokens, internal certificates.
CAP-05	Operational sabotage	Campaign blocking, distribution of inconsistent revocation policies, DoS on the deployment control plane.
CAP-06	Local debug access	Flash/RAM extraction, runtime patching, bypass of controls left unlocked.
CAP-07	Physical fault injection	Voltage/clock glitching, EMFI, disruption during critical boot checks.
CAP-08	Advanced reverse engineering	Firmware extraction, binary diff analysis, construction of target-specific payloads.

Table 3.3: Technical capabilities of the threat model

validated, tested, confirmed, rollback) must be logged in redundant metadata with CRC/ECC. This audit trail feeds both incident investigation and compliance evidence.

Update event log (full requirement). In addition to the technical boot flags, the device must maintain an event log dedicated to updates, transmissible over a **third-party channel** (telemetry, SOC agent, workshop collection), explicitly separate from the update transport channel. The objective is to prevent a disruption of the update channel from masking detection, audit or compliance signals.

Minimum content per event:

- a normalized event identifier (e.g. `DL_START`, `SIG_FAIL`, `ROLLBACK_DONE`);
- context identifiers (device, campaign, target version, A/B bank);
- a local timestamp (or a monotonic counter if no clock is available), result code and failure reason;
- a short artifact fingerprint (truncated hash), without disclosing any secret;
- the severity level and the action applied (retry, quarantine, rollback, resume).

Implementation constraints to respect:

ID	Objective	Verification criterion
OS-01	Image authenticity	Every active image is signed by an authorized, non-revoked key.
OS-02	Pre-execution integrity	Hash and manifest verified before any jump to a candidate slot.
OS-03	Anti-rollback	Installed version v such that $v \geq v_{min}$, with monotonic local state.
OS-04	Availability on update failure	A power cut during an update does not cause irrecoverable bricking.
OS-05	Privileged boot/runtime isolation	MPU/PMP policies forbid runtime writes to the RoT/metadata areas.
OS-06	Resistance to basic physical attacks	Detection of and response to simple glitches, production debug lockdown.
OS-07	Forensic traceability	Every boot state transition is logged and timestampable.
OS-08	Cryptographic agility	Key rotation/revocation without immobilizing the fleet.

Table 3.4: Normalized security objectives for MCU OTA

- **Limited MCU footprint**: a bounded ring log (explicit Flash/RAM budget), with a deterministic eviction policy and priority retention of critical events;
- **Robust writes**: append-only, atomic records, sequence numbers, CRC/ECC and power-fail-safe recovery to avoid silent corruption;
- **Strict access rights**: writes reserved to the privileged bootloader/update agent, restricted reads (authenticated maintenance, SOC), no arbitrary deletion by the application;
- **Integrity and authenticity**: local anti-tampering protection (MAC/batch signature, record chaining, local anchor) to detect post-incident falsification;
- **Confidentiality and minimization**: no key, secret, token or raw payload in the log; any personal data must be minimized and pseudonymizable;
- **Independent third-party channel**: asynchronous, buffered export to a channel separate from the update protocol (e.g. MQTT/TLS, secure syslog, factory channel), with acknowledgments and retries;
- **Retention and usability**: a retention window sized for forensic analysis, normalized event codes, and the ability to correlate with the campaign backend.

Associated verification criterion. An audit must be able to reconstruct, over a given period, the chronology *download* → *crypto verification* → *activation* → *confirmation/rollback*, even in case of a network outage on the main update channel.

3.4 Intrinsic limitations of the A/B approach

A/B does not fix:

- the absence of an immutable trust anchor;

ID	Security function	Implementation requirement
FS-01	Strict cryptographic verification	Mandatory manifest+image signature, versioned trust store, deny by default.
FS-02	Anti-rollback control	Monotonic counter (OTP/eFuse preferred), minimum version in local state.
FS-03	Transactional metadata management	Double copy, sequence numbers, CRC/ECC, append-only writes tolerant to power cuts.
FS-04	Privileged memory isolation	Locked MPU/PMP policies, boot/runtime separation, M-mode-only metadata or equivalent.
FS-05	Production debug lockdown	Locked lifecycle, controlled RMA, maintenance-opening log.
FS-06	Fault detection and response	Redundant checks, watchdog, consistency verification before the jump, recovery path.
FS-07	Publication key governance	HSM/KMS, separation of roles, rotation, tested emergency revocation.
FS-08	OTA forensic traceability	Immutable log of A/B transitions, campaign identifier, normalized failure reasons.
FS-09	Target compatibility check	Product/board/HW-revision verification, inter-image dependencies, explicit upgrade policy.

Table 3.5: Required security functions

- the compromise of the publication signing key;
- advanced physical attacks when no hardware mitigation exists.

Nor does it replace PKI governance and a secured CI/CD.

Scenario	Dominant capabilities	OS objectives	Required FS functions
Remote malicious firmware injection	AM-01 + CAP-01 / CAP-02	OS-01, OS-02, OS-08	FS-01, FS-07, FS-09
Downgrade and replay of a signed image	AM-01 / AM-03 + CAP-01 / CAP-06	OS-03, OS-07	FS-02, FS-03, FS-08
Publication chain compromise	AM-02 / AM-05 + CAP-03 / CAP-04	OS-01, OS-08	FS-01, FS-07, FS-08
A/B state metadata corruption	AM-03 / AM-04 + CAP-06 / CAP-07	OS-04, OS-07	FS-03, FS-04, FS-08
Secure boot bypass through fault injection	AM-04 + CAP-07 / CAP-08	OS-05, OS-06	FS-04, FS-05, FS-06
OTA campaign DoS	AM-01 / AM-05 + CAP-01 / CAP-05	OS-04, OS-07	FS-03, FS-08, FS-09

Table 3.6: Threats-objectives-functions traceability

4

Principle of the dual-bank (A/B) software architecture

4.1 Overview and operating principles

A robust implementation follows a three-stage chain:

1. Manufacturer ROM (immutable): minimal verification of the next stage.
2. Security bootloader (field-updatable or not): update policy, image verification, anti-rollback, bank selection.
3. Application: acknowledgment of a successful boot and runtime supervision.

This model is consistent with PSA RoT and DICE/RIoT for boot-derived identity [39, 45, 22].

4.2 Memory partitioning

Active bank (A) and passive bank (B). Recommended topology:

- a dedicated bootloader region, write-protected in production;
- two firmware slots of equal size (A and B) whenever possible;
- a persistent metadata area distinct from the slots, with a double copy.

This separation reduces the risk of cross-corruption between executable code and the boot decision state.

Bootloader area and persistent metadata. The metadata must include the version, the confirmation state, a retry counter, the image hash, the minimum required security level and the last failure reason. It must be written with a simple transactional protocol so that it remains reliably readable after a power cut.

Boot state storage and validation flags. Avoid unprotected single-bit flags. Prefer a redundant structure (double record + sequence number + CRC), written with an append-only protocol to limit Flash wear. This choice simplifies the detection of invalid states and makes the last valid boot transition explicit.

4.3 A/B update lifecycle

Download and storage in the inactive bank. The image is received in verified blocks (sized to the erase sector) with incremental hash verification, avoiding a second full read when RAM is constrained. The processing chain must also check the write bounds and the overall consistency before declaring the candidate bank complete.

Pre-activation cryptographic verification. The verifier validates at least:

- the manifest and image signature(s);
- the product/board identifier and the HW constraints;
- version monotonicity;
- the persistent data compatibility policy.

The acceptance policy must be identical between the backend and the device, to avoid decision divergences in production.

Controlled atomic switchover. The switchover must not rewrite large volumes within the critical window. Prefer selection through an active-slot pointer/flag, ideally in a dedicated, power-loss-resilient area. The operation must remain short, deterministic and easy to exercise in robustness campaigns.

Post-boot confirmation and rollback. The new image is marked *pending*. The application must issue a confirmation after health checks (drivers, connectivity, watchdog). Without confirmation within a window of N boots, an automatic rollback occurs. Confirmation criteria must remain stable over time, so that campaigns can be compared and quality drift identified quickly.

4.4 Secure boot state machine

Minimum states: *VALID A*, *TEST_B*, *CONFIRMED_B*, *ROLLBACK_A*, *RECOVERY*. Each transition is conditioned on local evidence (valid signature, pending flag, retry counter, watchdog state).

Model	**Principle**	**Advantages**	**Limits / preconditions**
Native dual-bank A/B	Two execute-in-place NVM banks, selection through a boot flag	Fast rollback, little runtime copying	Requires dual-bank NVM support and correct vector placement
Emulated A/B (single bank + external staging area)	Candidate image in external NOR, copy or XIP before activation	Works on MCUs without an internal dual bank	External bus attack surface, BOM cost, latency
Sector swap (MCUboot)	Progressive active/inactive exchange through a scratch area	Works on a single Flash	Metadata complexity, increased wear, corruption window
Overwrite-only + recovery image	Active slot overwritten, minimal recovery image kept aside	Reduced memory footprint	High bricking risk without a stable power supply

Table 4.1: Comparison of firmware update architecture models

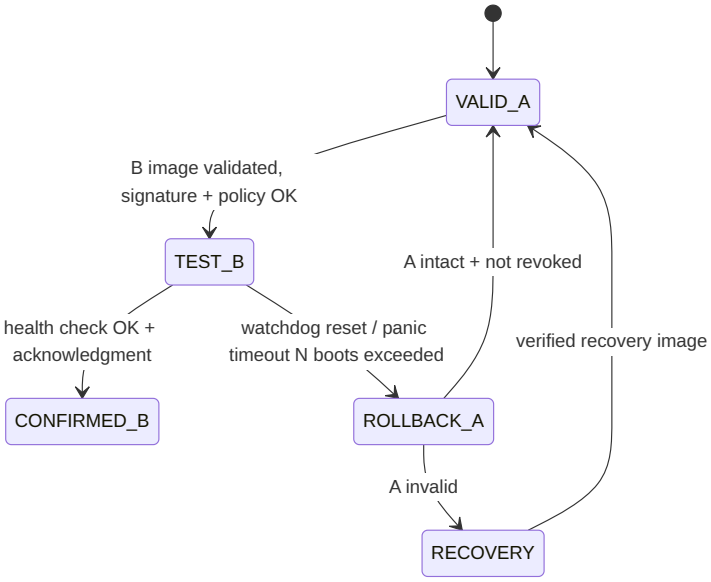


Figure 4.1: A/B update state machine

5

Specific constraints of MCUs without memory abstraction

5.1 Hardware constraints

Flash granularity, alignment and write atomicity. The *erase-size/program-size* pair directly conditions update safety. On many MCUs, erasure works per page/sector (1 to 128 KiB) while programming works per word or per line. This mismatch imposes a strict transactional strategy:

- never rewrite a critical structure in place (metadata, counters, boot flags);
- write a complete new record, then commit it with a final marker;
- validate consistency and integrity with CRC/ECC before use.

NVM wear and operational cycle budget. Internal Flash cells typically withstand 10k to 100k cycles; some industrial profiles impose severe temperature/retention margins. The minimum prerequisites are:

- an *append-only* journal for boot and anti-rollback states;
- explicit wear leveling (slot ring, page rotation, bounded collection);
- an endurance budget computed over the product lifetime (cycles/day x years);
- correlated endurance and power-cut tests [43, 36].

Often-overlooked hardware security prerequisites. For an A/B design to be defensible, in practice you need:

- an immutable anchor (ROM, OTP, eFuse) for the root key or its hash;
- a production debug lockdown mechanism with a separate RMA procedure;
- a power source sized for the critical write windows (or a supercap);
- an independent watchdog configured from the very first boot steps.

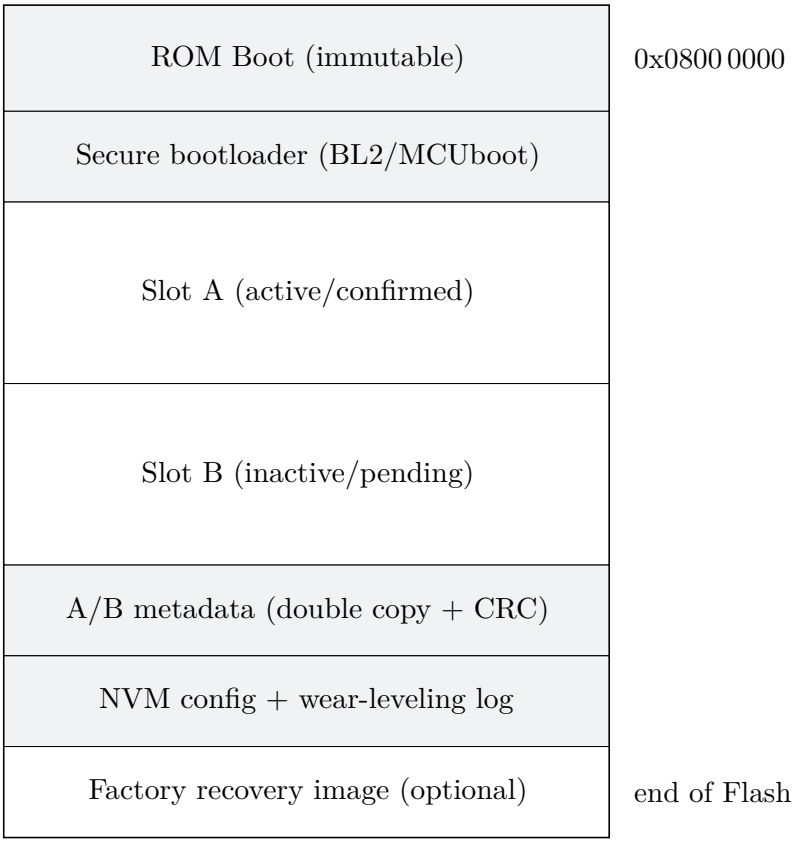


Figure 5.1: Example A/B memory map on an internal-Flash MCU

Limited RAM and impact on cryptographic verification. Verifying a post-quantum signature can exceed the RAM budgets of entry-level MCUs. Viable strategies are:

- streaming hash + signature verification on a compact manifest;
- offline verification of blocks with a condensed Merkle proof;
- a hybrid classic + PQC approach reserved for certain product profiles.

Boot time and availability window. A long boot increases the DoS surface. The verifier must be time-bounded, with a watchdog, a defined retry threshold and a deterministic exit path to a known-valid image. Define a measurable *secure boot budget* (P95/P99) and SOC alarm thresholds.

5.2 Software constraints

Fixed addressing, boot vector and PIC. Without memory abstraction, the application is often linked for one specific base address. To understand the problem, three notions must be distinguished:

- **Absolute-address code**: the compiler/linker emit references to fixed addresses (e.g. a function at `0x08020000`, a data item at `0x20001000`).
- **PIC** (*Position-Independent Code*): the code favors relative (PC-relative) computations, so that it can be placed at several base addresses.

- **PIE** (*Position-Independent Executable*): a binary fully relocatable at load time; on MCUs, a complete PIE is rarely used, for lack of a rich loader such as on a general-purpose OS.

In a **dynamic incremental update** scheme (functions added, removed or remapped at runtime), PIE becomes necessary: the list of functions is no longer fully frozen at production time. The runtime must then manage a dynamic mapping in RAM, with a loader able to build and update a consistent GoT at load time to resolve global symbols and pointers.

Concretely, when the compiler generates absolute accesses, the firmware becomes coupled to **a single slot address**. If that same binary is copied into the other bank, jumps and data accesses may point to the wrong area, making the dual bank unstable or unusable. This is the main reason strictly non-PIC firmware is incompatible with an A/B architecture.

The role of the GOT (*Global Offset Table*) is to avoid freezing global addresses into each instruction by centralizing adjustable offsets/pointers. On a microcontroller, a complete GOT can be costly in Flash size, RAM and latency; a compromise is therefore often chosen.

To support A/B, three strategies exist:

- two separate builds (A and B) with fixed addresses;
- a single, partially relocatable build (position-independent code/PIC);
- a minimal trampoline + dynamic VTOR pointing to the chosen slot's vector table.

Full PIC on Cortex-M remains costly in size and performance. In practice, a *PI-lite* compromise (relocatable critical sections, limited absolute calls) is often chosen.

Related design prerequisites:

- linker scripts that are versioned and continuously tested (A/B, secure/non-secure, debug/prod);
- explicit invariants on the vector table, fault handlers and low-level init;
- non-regression tests on memory map migration between versions.

RV32 specifics with PMP. On RV32, update robustness strongly depends on the quality of the PMP partitioning [18]. The minimum requirements are:

- the ROM/boot region configured execute-only or read-execute, and locked (`L`=1);
- the A/B metadata region read-only outside M-mode;
- runtime prohibition of any write to the ranges holding the public key, the anti-rollback policy and the verification code;
- explicit re-initialization of `pmpcfg/pmpaddr` at each privilege transition.

In addition, the audit prerequisites must include:

- proof of PMP range coverage (no unintended executable hole);
- verification of the `M`→`S/U` transitions and the `mtvec` exception paths;
- a resistance test against `ecall`/interrupt abuse at the privilege boundary.

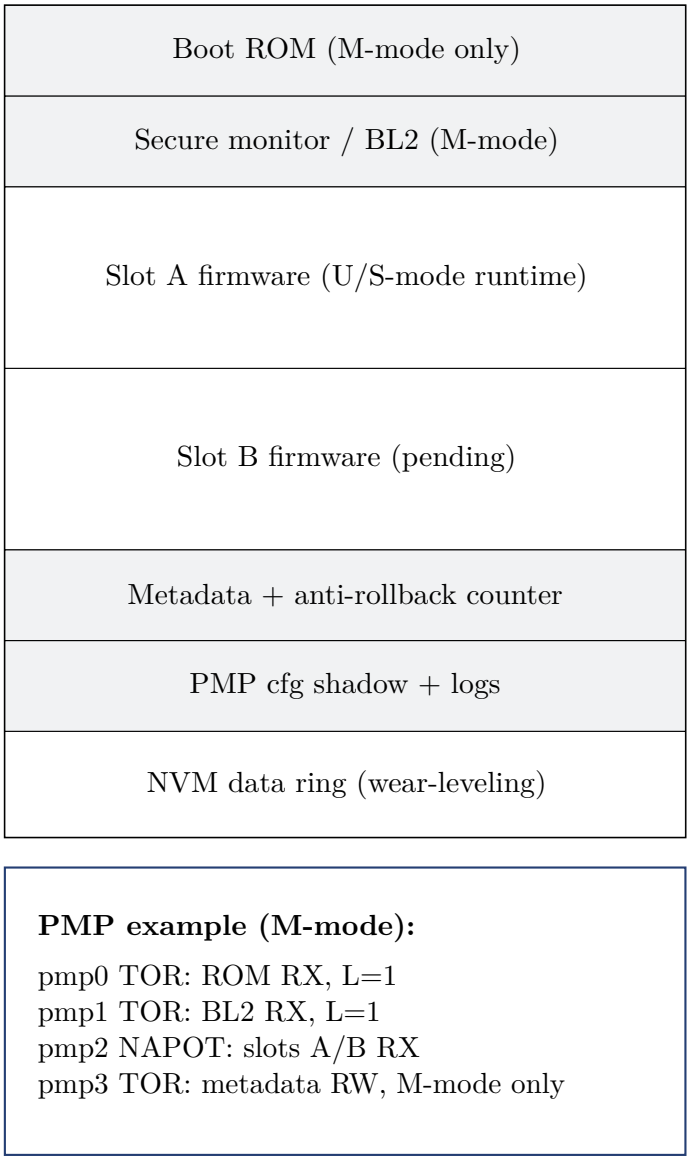


Figure 5.2: Example RV32 memory map with PMP partitioning

Low-level driver dependencies. The clock, Flash controller, MPU/SAU and debug lockdown drivers must be compatible with both banks and initialized deterministically; otherwise a rollback may fail before serial output is even available.

Inter-bank compatibility and data migration. The application state (configuration NVM) must be versioned. Any irreversible migration must be deferred until the new firmware is confirmed; otherwise rollback becomes impossible.

Migration prerequisites:

- a versioned schema with `vN`→`vN+1` and, when possible, `vN+1`→`vN` transformers;
- an atomic migration marker distinct from the boot confirmation marker;
- cross-compatibility guardrails between application, bootloader and manifest.

5.3 Operational constraints

Power loss during an update. Every critical write must be broken down into idempotent transactions. Forbid unlogged metadata modifications.

Partial updates forbidden or strictly controlled. Binary deltas save bandwidth but increase the risk of base image desynchronization. Their use must be reserved for strictly inventoried fleets.

Minimum conditions for accepting a delta:

- an explicit fingerprint of the base image;
- post-application verification with a full hash of the reconstructed image;
- automatic fallback to a full image on any mismatch.

Version management and anti-rollback. The anti-rollback mechanism must be designed according to the available hardware:

- a monotonic OTP/eFuse counter (ideal);
- a counter in a protected Flash area with anti-tearing and cross-checking;
- a strict server-side policy if no hardware root exists (lower trust level).

A fundamental prerequisite is a consistent versioning policy:

- a single version semantics (bootloader, app, secondary components);
- authorized upgrade windows;
- documented exception rules (critical fix, emergency rollback).

HW constraint	Available option	Security impact	Recommendation
Internal dual-bank NVM	Yes	Simplified atomic activation	Prefer native A/B, deferred confirmation
Internal dual-bank NVM	No	More copying/swapping, power-cut risk	Add an external staging area or a robust recovery mode
Immutable secret storage	OTP/eFuse	Reliable anchor, strong anti-rollback	Store the root key hash + a monotonic counter
Immutable secret storage	None	High exposure to physical reflash	Compensate with an enclosure, debug lockdown, frequent attestation
Debug protection	Irreversible lock (RDP2, lifecycle)	Reduces key extraction and firmware dumps	Enable in production with a dedicated RMA procedure

Table 5.1: Structuring hardware constraints for OTA security

6

Security requirements on the update process

6.1 Organizational prerequisites

Organizational prerequisites must be explicit and auditable.

PR-ORG-01 – Separation of roles and approval workflow. Apply dual control to the publication signature: build, security approval, signing and publication, each with a distinct account/service account and an action log.

PR-ORG-02 – Key and secret governance. Keep signing keys in an HSM/KMS, enforce usage policies (MFA, quorum, rotation, expiration), and run key compromise exercises.

PR-ORG-03 – Evidence and auditability. Retain reproducible build evidence, artifact fingerprints, signing logs, SBOMs and campaign metadata [42, 31].

6.2 Device-side technical prerequisites

PR-DEV-01 – Immutable Root of Trust. The verification root key must be anchored in ROM, OTP or eFuse. Any update of this anchor must go through an exceptional, authenticated and logged path.

PR-DEV-02 – Privilege isolation (Cortex-M vs RV32). To satisfy OS-05:

- on Cortex-M: strict MPU protection of the boot + metadata areas, and a separate execution policy between critical handlers and the application;
- on RV32: the PMP defined in M-mode with locking, and the application running in a lower-privilege mode with mediated service calls.

Recommended reference framework for the MPU-PMP objectives/countermeasures: [18].

PR-DEV-03 – Secure boot and image verification. The boot verifier must be minimal, auditable, and resistant to simple faults: redundant verification, control-flow checks, and consistency checks before the jump.

PR-DEV-04 – Anti-rollback protection. Anti-rollback must combine the manifest version with monotonic local state. Without local state, an attacker can replay an old signed binary.

PR-DEV-05 – Safe and deterministic rollback. Roll back only to a previously confirmed, non-revoked image.

PR-DEV-06 – Power robustness and watchdog. The system must define: recovery points after a power cut, a confirmation timeout, and a watchdog strategy during the activation phase.

6.3 Backend-side technical prerequisites

PR-BE-01 – Controlled signing CI/CD. A hardened integration chain: provenance, SBOM, dependency scanning, build signatures and publication artifacts.

PR-BE-02 – Authenticated artifact distribution. Mutual TLS for critical channels, backend certificate pinning, and application-level signatures independent of the transport.

PR-BE-03 – Operable key revocation and rotation. Plan for revoking a compromised publication key without immobilizing the fleet (a versioned list of authorized keys in the manifest and a transition policy).

6.4 Update acceptance criteria

An update may be promoted only if:

- the full cryptographic verification succeeded;
- hardware compatibility is declared and validated;
- anti-rollback is satisfied;
- the post-boot health test passes;
- confirmation telemetry is received within the expected window.

7

Cryptographic trust chain of the update

7.1 Cryptographic objectives

Authenticity. Guarantee that the image comes from an authorized entity.

Integrity. Guarantee that payloads and metadata have not been altered.

Freshness and anti-replay. Guarantee that an old artifact cannot be reinstalled outside the policy.

Non-repudiation. Retain auditable signing evidence without exposing the private keys.

Crypto reference frameworks to apply. The cryptographic policy must be aligned with the following reference frameworks:

- **NIST**: FIPS 180-4, FIPS 186-5, SP 800-57 (key lifecycle management), SP 800-193 (platform firmware resiliency), FIPS 203/204/205 for the PQC transition [27, 32, 29, 28, 34, 33, 35];
- **ANSSI**: recommendations on cryptographic mechanisms and key/certificate management policies in a national/regulated context [1, 3, 17];
- **Europe**: product security requirements stemming from the Cybersecurity Act and the Cyber Resilience Act (CRA), to be broken down into OTA controls and vulnerability governance [12, 13].

7.2 Image format and signed manifest

The embedded state of the art converges on SUIT CBOR + COSE to describe and verify firmware updates [24, 25, 41].

Minimum manifest requirements. The manifest must include at least:

- the product identifiers, PCB revision, SKU and hardware constraints;
- the version, monotonicity, upgrade policy and validity window;
- the fingerprints of all components (app, secure world, radio stack, secondary bootloader);
- the inter-image dependency constraints and the minimum required security level.

7.3 Recommended algorithms

Hash and KDF. SHA-256/384 remains the pragmatic baseline. For local key derivation, HKDF is recommended, with an explicit context and domain separation [27, 20].

Classic signatures (pre-PQC). Ed25519 offers a good compromise on MCUs; ECDSA P-256 remains dominant for industrial compatibility and certification [32, 7].

Payload encryption (optional). OTA encryption is not a substitute for signing. It is relevant for IP protection and confidentiality in transit and at rest (e.g. AES-GCM), at the cost of additional key provisioning/rotation complexity.

7.4 Key management

Root anchoring. Store the root key hash (or a compact root certificate) in an immutable area.

Firmware signature delegation. Use short-lived intermediate keys, with an explicit delegation policy.

Rotation, expiration, revocation. Design transition manifests that authorize the old and the new key simultaneously over a limited window, then force the removal of the old one.

Key/certificate governance prerequisites.

- an inventory of the active keys and their exact usage;
- a periodic rotation policy plus an emergency rotation;
- proof of a revocation exercise at least once a year;
- a secure decommissioning process for obsolete keys.

7.5 Concrete example: the secure update mode of the WooKey project

The WooKey project (ANSSI) is an interesting example because it combines a standard USB DFU transport with a product-specific cryptographic and operational overlay [4]. The update chain is not limited to a late signature verification: the platform maintains strong operator authentication through an external token, and actively involves the DFU token in the derivation of session keys while the firmware blocks are being written.

In this architecture, the update format includes a signed header (ECDSA), version metadata, an IV and HMAC protection of the header; the body is block-encrypted (AES-CTR) to reduce the malleability exploitable in case of fault. This matters on RAM-constrained MCUs, where the image is often written to Flash before the final signature verification. WooKey also adds defense in depth on the DFU mode (task isolation and separation of roles), together with a dual-bank flip-flop mechanism enforcing strict anti-rollback at boot [4].

8

Protocols for updating microcontrollers

8.1 Objective and boundaries of an update protocol

An update protocol primarily defines *how* to transfer and drive an image (state, blocks, resume, status), but it does not automatically guarantee *who* is authorized nor *what* is legitimate. In practice, a complete cryptographic policy must be layered on top: producer authenticity, image integrity, anti-replay, anti-rollback, and usable logging.

8.2 USB DFU: a transport and orchestration standard

USB DFU (Device Firmware Upgrade) standardizes a state machine and a set of control requests for firmware upload and management (`DFU_DNLOAD`, `DFU_UPLOAD`, `DFU_GETSTATUS`, `DFU_CLRSTATUS`, `DFU_ABORT`) [48]. Its industrial appeal is its compatibility with existing tools and workshop maintenance flows.

Native DFU limitations. By itself, the DFU protocol mandates neither firmware signing, nor payload confidentiality, nor an anti-rollback policy. Nor does it define key governance or complete forensic traceability. Without a security overlay, DFU therefore remains a transport/-command mechanism exposed to replay, unauthorized images or host workstation compromise.

The WooKey example on top of DFU. WooKey keeps DFU interoperability, but adds an application-level cryptographic layer: strong authentication, token-based session key derivation, block encryption, ECDSA signature, header HMAC and consistency verification at boot. This approach illustrates a sound general practice: keep a standard transport protocol, while moving the security guarantees into an explicit cryptographic framework [4, 48].

The state machine below reproduces the USB-IF class DFU states (application branch and DFU branch) and the main operational transitions used by update tools (DETACH, DNLOAD, UPLOAD, GETSTATUS, ABORT, CLRSTATUS, reset) [48].

8.3 SCP03 (GlobalPlatform): a secure channel for sensitive commands

SCP03 is a secure channel protocol defined by GlobalPlatform to protect APDU commands sent to a secure component (SE/eSE/UICC) [16]. It relies on static domain keys (typically ENC/MAC/DEK), a host-card random exchange, then a session key derivation. Depending on the configured security level, commands can be MAC-protected (C-MAC), encrypted (C-ENC), and responses authenticated (R-MAC).

Useful properties for MCU updates. SCP03 provides confidentiality, integrity and anti-replay protection on the command channel. It is particularly relevant for provisioning secrets, driving security states, or encapsulating critical operations (image activation, key rotation, or deployment of firmware decryption keys).

Scope limits. SCP03 is not, by itself, an end-to-end firmware distribution protocol: it replaces neither a manifest specification, nor software dependency handling, nor a product anti-rollback policy, nor the A/B recovery logic. It must be integrated into a broader architecture (secure boot, version metadata, transactional state, telemetry).

8.4 DFU and SCP03 comparison

Option	Strength	Main limitation	Recommended use
Native USB DFU	Interoperability and broad tooling	No native product-level crypto guarantees	Workshop maintenance, prototyping, transport baseline
DFU + crypto overlay (e.g. WooKey)	Compatibility + security applied to the firmware	Integration complexity (format, keys, token, states)	Exposed products, high resilience requirements
SCP03 (GlobalPlatform)	Strongly protected command channel (MAC/encryption/anti-replay)	Does not cover the complete OTA/A-B logic on its own	Secure provisioning, critical commands, secrets management

Table 8.1: Positioning of update protocols and secure channels

8.5 Integration recommendations for MCUs

For a robust implementation, the following combination is recommended:

- a standard transport protocol (DFU, HTTP(S), BLE OTA) for operability;
- a signed manifest (SUIT/COSE) for cryptographic eligibility;
- a hardened command channel (SCP03 or equivalent) for sensitive operations;
- a transactional A/B state machine for recoverability;
- security telemetry and traceability for SOC operations and compliance.

Matching the protocol to the interface. The choice of standard protocol must remain consistent with the communication interface actually available on the device. For example, HTTP or TFTP suit IP/Ethernet (or Wi-Fi) network stacks, but are not, as such, the right choice on low-speed serial buses such as SPI or I2C, which generally call for a lighter transport protocol or a suitable gateway.

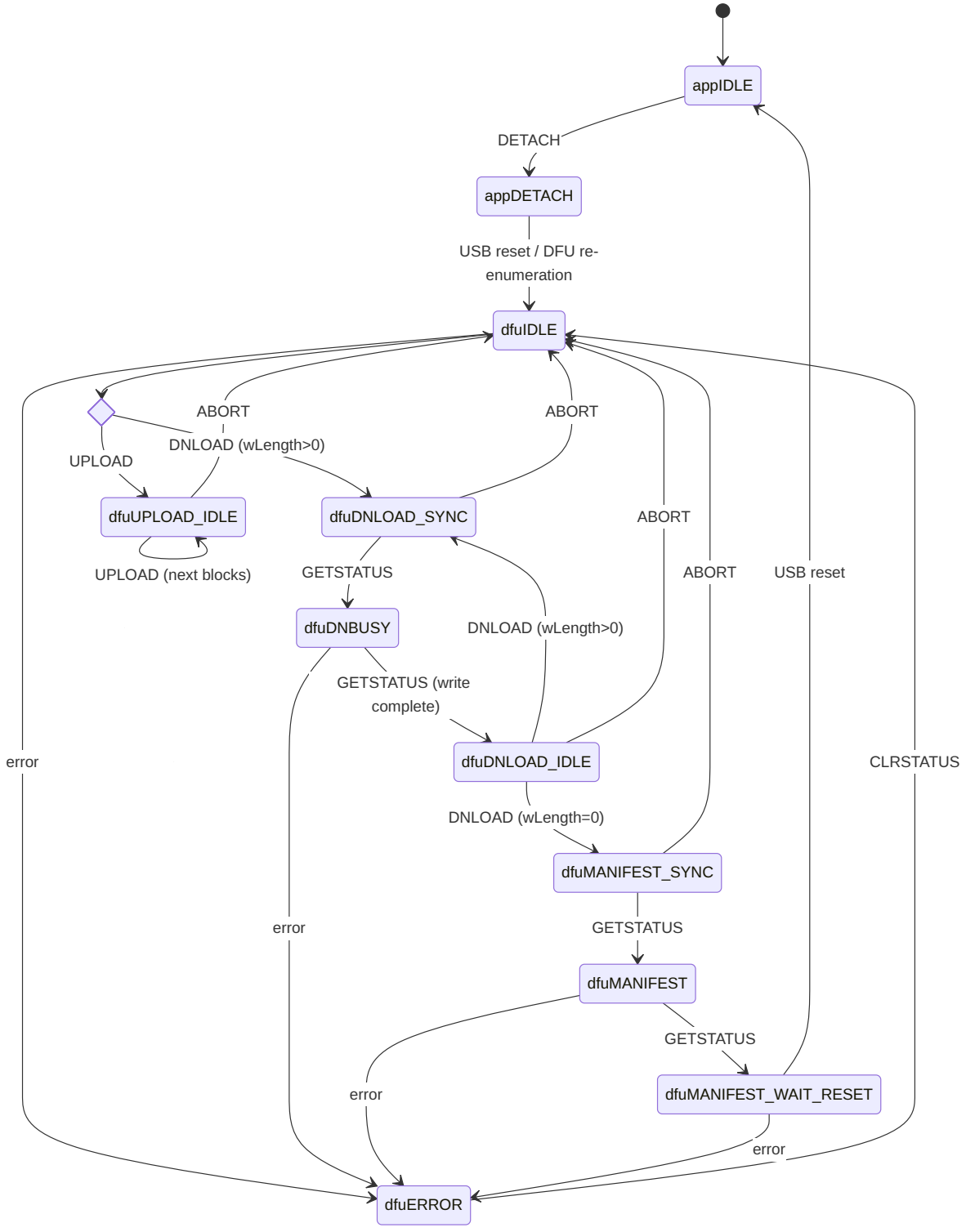


Figure 8.1: USB DFU protocol state machine (USB-IF DFU class)

9

PQC and a crypto-agile transition strategy

9.1 Why anticipate PQC

The *harvest now, decrypt/forge later* risk mainly concerns long-lived assets. For critical firmware with a 10–20 year lifecycle, a PQC roadmap is relevant [30, 34, 33, 35].

9.2 PQC use cases in the update chain

Post-quantum firmware signatures. ML-DSA (Dilithium) is the main candidate on the standardization side, but its key and signature sizes can penalize the most constrained MCUs. A realistic strategy is to reserve its initial use for product lines with sufficient memory headroom.

Post-quantum key exchanges (where applicable). If the provisioning channel requires a secret exchange, ML-KEM (Kyber) can be introduced first on gateways or on more capable modules. On tightly constrained MCUs, a hybrid architecture delegating part of the operations to an adjacent component is often more industrially tenable.

9.3 Hybrid classic + PQC approach

Dual signature and verification policy. The recommended approach combines a mandatory classic signature with an experimental PQC signature, then gradually reverses the priorities as the ecosystem matures. This coexistence must be explicit in the manifest and in the backend rules.

Memory, performance and boot latency impacts. The main cost is the memory footprint and the boot-time verification. It is often preferable to verify the dual signature during the activation phase, then keep a lighter nominal reboot path once the image is confirmed.

Progressive migration plan and controlled rollback. Define stages (pilot, coexistence, generalization, classic decommissioning) and associate with each of them success criteria, alert thresholds and a rollback mechanism testable in near-real conditions.

9.4 Crypto-agility requirements

Versioning of cryptographic suites. The manifest must carry a versioned crypto suite identifier that remains stable and interpretable over the entire product lifetime.

Backend- and device-side negotiation and policy. The backend must never impose a suite that is not supported locally; the device exposes a signed capability profile. This negotiation must be deterministic and rejected on any profile inconsistency.

Managing algorithmic obsolescence. Plan an end-of-life date for algorithms, together with forced migration campaigns, to avoid fleets stuck in lasting crypto debt.

10
Device enrollment and CA hierarchy

10.1 Industrial PKI model

Offline root CA. Keep the trust root offline and limit its use to exceptional issuances. This discipline greatly reduces the impact of an operational compromise on the PKI as a whole.

Intermediate CAs per usage (production, test, RMA). Strictly separate the domains to avoid cross-contamination. Lifecycle policies and issuance rights must be independent across environments.

Registration authorities and identity control. The RA must bind the hardware identity (serial number, batch, SKU) to the cryptographic identity, and retain the verification evidence for audit and investigation.

10.2 Reference schema: CA hierarchy, enrollment and key derivation

Value of sub-CAs. Sub-CAs segment risks and responsibilities. A compromise in one domain (for example RMA) must not make it possible to sign production firmware. Rotation and revocation also become more targeted, without mass re-issuance across the fleet or direct impact on the offline root.

Enrollment mode (PKI) versus secret injection mode. Enrollment assigns each device its own cryptographic identity (certificate, traceability, granular revocation). Conversely, injecting a shared static secret sometimes simplifies manufacturing, but widens the blast radius of a leak: a single compromise can affect a whole batch, or even a product family.

Use of key derivation. Derivation (HKDF or equivalent) produces distinct per-usage keys (attestation, update session, command channel) from a hardware root secret and a context (device identifier, version, counter/nonce). This limits the reuse of long-lived keys, reduces lateral movement and eases logical rotation without full reprovisioning.

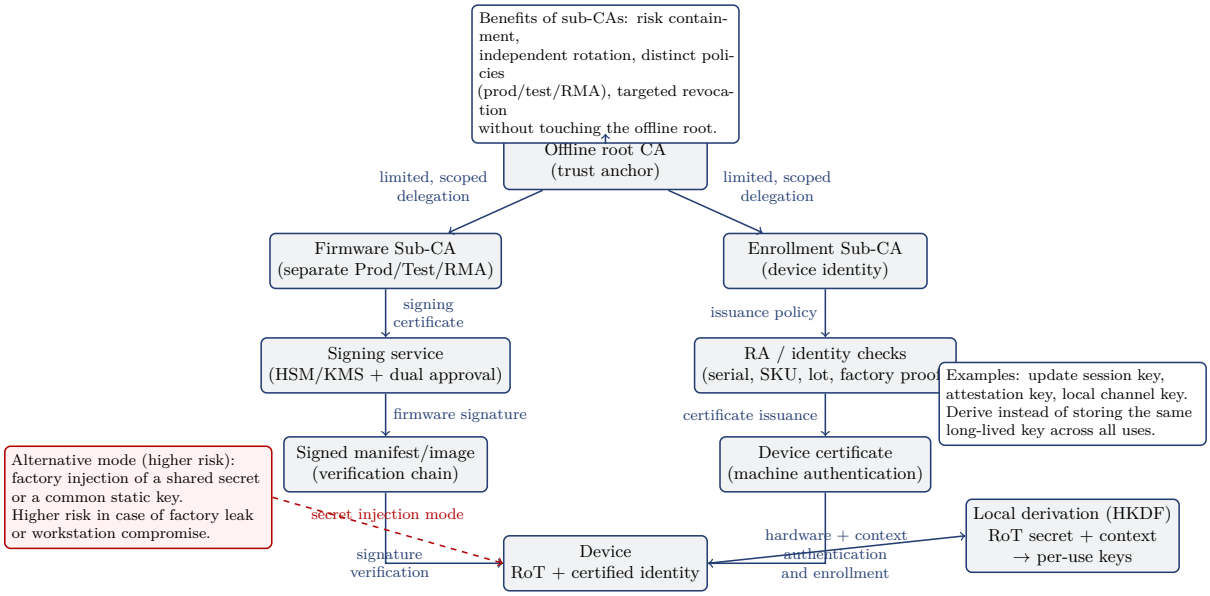


Figure 10.1: Typical CA hierarchy for firmware signing and device enrollment

10.3 Initial enrollment process

Identity injection at manufacturing. Perform the injection in a secured environment, with sealed operation records and personalization evidence. The supply chain must be able to demonstrate who injected what, when, and under which authorization.

Certificate and trust anchor provisioning. Prefer short-lived, renewable certificates, with a compact embedded profile suited to the MCU's memory constraints.

Secure personalization per batch or per unit. Per-unit personalization improves traceability but carries a higher industrial cost; per-batch personalization requires stricter compensating controls to maintain the same level of trust.

10.4 Certificate lifecycle

Renewal and rotation. Renewal must be anticipated ahead of expiration to avoid fleets falling into crypto debt. Emergency rotation must be tested regularly against realistic degraded scenarios.

Revocation (CRL/OCSP or an equivalent embedded mechanism). On intermittently connected MCUs, revocation can be distributed through a policy embedded in the campaign manifests. The goal is measurable revocation coverage, even with intermittent connectivity.

Handling compromised or decommissioned devices. Switch to quarantine mode, cut access to sensitive services and mark the identity as retired, to prevent any unauthorized reuse.

11
Attestation and software compliance evidence

11.1 Attestation objectives

Proof of integrity of the active firmware. The attestation must demonstrate that a measured fingerprint corresponds to an approved firmware and a known configuration. This proof only becomes usable if it is compared against a controlled reference on the trust service side.

Proof of version and security state. The report must include at least the patch level, the debug status and the active secure boot policy, in order to support an explicit, reproducible access decision.

11.2 Local and remote attestation

Boot measurements (secure measurements). Measure the bootloader, the application and the security configuration within a chain of trust, using a format that stays stable over time to ease verification automation.

Signed attestation report. Sign the report with an attestation key derived from the RoT (DICE/RIoT) and associate a freshness nonce [45, 22, 8]. The report structure must remain lean so that it stays compatible with constrained devices.

Verification on the trust service side. The service validates the certificate chain, the nonce freshness and the acceptance policy before granting access to sensitive resources.

11.3 Linking attestation to operational decisions

Conditional network access. Access to critical APIs must be conditioned on a compliant, recent attestation, according to an explicit risk policy.

Triggering remediation or quarantine. On non-compliance, apply graduated remediation (privilege reduction, forced update, isolation) to limit the operational impact without losing

security control.

12
Secure A/B operations in real-world conditions

12.1 Deployment strategies

Canary, progressive waves, maintenance windows. Campaigns must start on a representative sample, then progress in waves with automatic stop thresholds. This deployment mode limits the scale of an incident and speeds up its detection.

Version promotion criteria. Promote a version only if the confirmed boot rate is stable, with no abnormal rollbacks and no security incident over the defined observation window.

Heterogeneous fleet management. Segment by SKU, board revision, radio region and crypto capabilities, to avoid global decisions ill-suited to some technical segments.

12.2 Security monitoring and telemetry

Key indicators (boot success, rollbacks, verification failures). Collect at least:

- the failed cryptographic verification rate;
- the automatic rollback rate;
- the average activation→confirmation latency;
- the distribution of failure reasons.

These indicators must be historized and compared per fleet segment, so that a local drift is detected quickly.

Anomaly detection and SOC alerting. Massive signature rejection patterns can indicate an attempted malicious campaign. Alert rules must distinguish quality defects from attack signals, to reduce operational noise.

12.3 Update-related incident response

Campaign blocking and emergency revocation. Provide a campaign kill switch and a revocation list that can be distributed quickly. The conditions for triggering and lifting the block must be documented in advance.

Forensics and evidence collection. Retain manifests, boot logs, campaign identifiers and signing evidence for post-incident investigation. Their probative value depends on timestamp quality and trace integrity.

13
Verification, validation and compliance

13.1 A/B test requirements

Robustness tests (power cuts, Flash corruption). Inject power cuts at every critical step and verify that no bricking occurs. The results must be reproducible across several hardware batches.

Security tests (invalid signature, forced rollback, replay). Run a systematic negative test suite, including controlled fault injection, to validate the fail-closed behavior in adversarial scenarios.

Performance tests (boot time, update time). Define maximum budgets and validate them under degraded thermal and voltage conditions, to guarantee sufficient operating margin in industrial environments.

13.2 Security criteria before production release

Production release requires:

- a bootloader code audit;
- evidence of security non-regression testing;
- validation of the revocation/rotation procedures;
- a publication key compromise simulation.

13.3 Normative traceability and regulatory requirements

An explicit mapping must be maintained to ETSI EN 303 645, NISTIR 8259A, IEC 62443 and the ANSSI recommendations for embedded/IoT [11, 14, 19, 2].

14

Implementation recommendations and anti-patterns

14.1 Design best practices

- A minimal bootloader, code reviews, few dependencies.
- Transactional metadata with CRC/ECC and sequence numbers.
- Strict key separation (development, pre-production, production).
- A hardware-anchored anti-rollback policy.
- Secure telemetry geared towards early detection.

14.2 Common mistakes to avoid

Implicit trust in the network transport. TLS without an application-level signature exposes you to infrastructure compromises and is not enough to guarantee the legitimacy of a firmware image.

No operable revocation policy. Without operational revocation, a key compromise quickly becomes catastrophic, because the fleet cannot be sanitized within an acceptable timeframe.

Uncontrolled rollback. Allowing free downgrades cancels out vulnerability remediation efforts and reintroduces versions known to be weak.

Excessive coupling between bootloader and application. Tight coupling complicates manifest format migrations, slows down patching and increases the risk of regression as the platform evolves.

14.3 Operational security checklist

Control	Target status	Expected evidence
Immutable trust anchor (OTP/eFuse/ROM)	Mandatory	Hardware documentation + fuse readout in production
Signature verification + anti-rollback at boot	Mandatory	Negative test report and rejection logs
Automatic power-fail-safe rollback mechanism	Mandatory	Power-cut test campaign
Operable key revocation and rotation	Mandatory	Exercised procedure + exercise report
Centralized update security telemetry	Strongly recommended	SOC dashboard + event retention
PQC migration preparation (hybrid mode)	Recommended	RAM/Flash/latency impact study + roadmap

Table 14.1: Minimum security checklist for MCU updates

15
Overview of modern attacks on MCU updates

15.1 State of the art of attacks

The most relevant attack families today are:

- **Supply chain**: compromise of artifacts or of the integration chain;
- **Downgrade/replay**: replay of legitimate but vulnerable images;
- **Fault injection**: voltage/clock glitching, EMFI to bypass checks;
- **Debug abuse**: SWD/JTAG reactivation, firmware dumps, runtime patching;
- **TOCTOU**: verification performed on an object different from the one executed;
- **Metadata desynchronization**: corruption of the confirmation flags.

Attack-to-OS-objective mapping. To guide verification engineering:

- malicious firmware injection → OS-01, OS-02, OS-08;
- downgrade/replay → OS-03, OS-07;
- glitch/fault injection → OS-05, OS-06;
- metadata corruption/power loss → OS-04, OS-07;
- publication key compromise → OS-01, OS-08.

15.2 Attacks vs. controls matrix

Attack	Vector	Impact	Priority controls
Malicious firmware injection	Compromised integration chain/repository	Arbitrary code execution	Offline signing, separation of roles, build provenance, fast revocation
Signed image downgrade	Local/remote replay	Reopening of known CVEs	OTP/eFuse monotonic counter, minimum version policy
Verification bypass through glitching	Physical access, fault injection	Secure boot bypassed	Redundant checks, fault detectors, anti-tamper enclosure, debug lockdown
Boot metadata corruption	Power cut or Flash write attack	Undefined boot state/-DoS	Append-only logging, double copy + CRC, strict state machine
Publication key compromise	IT secrets leak	Fleet takeover	HSM/KMS, emergency rotation, versioned trust store, attestations

Table 15.1: Summary matrix of MCU OTA attacks and countermeasures

16
Conclusion

16.1 Summary of the guarantees provided by the dual bank

The A/B architecture provides strong operational resilience against update failures and a sound foundation for applying modern cryptographic controls.

16.2 Conditions for maintaining security over time

Security remains conditioned on key governance, publication discipline, and the ability to refresh the anti-rollback and revocation policies.

16.3 Hardening roadmap (including PQC)

Recommended priorities:

1. Strong hardware anchoring (OTP/eFuse) + robust rollback.
2. SUIT/COSE manifest standardization + SOC-usable security telemetry.
3. Hybrid classic/PQC introduction on capable product families.
4. Generalization of remote attestation for fleet risk management.

A

Appendix A – Boot state machine example

Suggested minimum states:

- `BOOT_A_CONFIRMED`
- `BOOT_B_PENDING`
- `BOOT_B_CONFIRMED`
- `ROLLBACK_TO_A`
- `RECOVERY_MODE`

Critical transitions:

- pending → confirmed only after proof of application health;
- pending → rollback on timeout, watchdog reset, or security assertion;
- rollback → recovery if the previous bank is invalid.

B

Appendix B – Minimum signed manifest requirements

The manifest must contain at least:

- the product identifier and hardware revision;
- the firmware version and the anti-rollback policy;
- the image's cryptographic fingerprint;
- the authorized verification key(s) and their validity period;
- the inter-component dependencies and installation preconditions.

C

Appendix C – Standard key and certificate management policy

Typical policy:

- an offline root, segregated online intermediates;
- periodic rotation, half-yearly or yearly depending on criticality;
- a revocation exercise at least quarterly;
- immutable logging of issuances and withdrawals.

D

Appendix D – Threat matrix and associated controls

The complete matrix must link each threat to:

- prevention controls;
- detection controls;
- remediation controls;
- the associated test evidence and the control owner.

Bibliography

[1] ANSSI. Recommandations de securite relatives aux mecanismes cryptographiques. Referentiel ANSSI, 2020.

[2] ANSSI. Recommandations de securite relatives a l'iot et aux systemes embarques. Guides et recommandations ANSSI, 2023.

[3] ANSSI. Referentiel general de securite (rgs). Reglementation identite et confiance numerique, 2025. https://cyber.gouv.fr/reglementation/reglementation-identite-confiance-numerique/securite-echanges-voie-electronique/referentiel-general-de-securite/, accessed 2026-07-04.

[4] ANSSI WooKey Project. Wookey architecture: Cryptography, dfu mode and flip-flop mechanism. Project documentation, 2019. https://wookey-project.github.io/architecture.html, accessed 2026-07-04.

[5] Anvil Secure. Glitching stm32 read-out protection with voltage fault injection. Technical report, Anvil Secure, 2024.

[6] Arm Ltd. Arm platform security architecture firmware framework. https://developer.arm.com/architectures/security-architectures/platform-security-architecture, 2023. Accessed 2026-07-04.

[7] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. In *Cryptographic Hardware and Embedded Systems (CHES)*, 2011.

[8] H. Birkholz et al. Entity attestation token (eat). IETF RFC 9711, 2024.

[9] CISA and NCSC. Solarwinds and sunburst: Lessons for software supply chain security. Public advisories and post-incident reports, 2021.

[10] Dragos Inc. Trisis: Malware that attacks safety instrumented systems. Technical report, Dragos, 2017.

[11] ETSI. Cyber security for consumer internet of things: Baseline requirements. ETSI EN 303 645, 2020.

[12] European Union. Regulation (eu) 2019/881 (cybersecurity act). Official Journal of the European Union, 2019.

[13] European Union. Regulation (eu) 2024/2847 (cyber resilience act). Official Journal of the European Union, 2024.

[14] M. Fagan, K. Megas, K. Scarfone, and M. Smith. Iot device cybersecurity capability core baseline. NISTIR 8259A, 2020.

[15] Nicolas Falliere, Liam O Murchu, and Eric Chien. W32.stuxnet dossier. Technical report, Symantec, 2011.

[16] GlobalPlatform. Secure channel protocol '03' – amendment d v1.2. GlobalPlatform Secure Element specification, 2020. Specification listing: https://globalplatform.org/specs-library/secure-channel-protocol-03-amendment-d-v1-2/, accessed 2026-07-04.

[17] Gouvernement francais. Instruction interministerielle no 901. Instruction interministerielle relative a la protection des systemes d'information sensibles, 2013.

[18] Hardware Hacking Laboratory. H2lab-stig-mpu-001: Guide technique de protection memoire physique pour systemes embarques. STIG H2LAB, 2026. https://blog.h2lab.org/stigs/nouveau-guide-technique-protection-memoire-physique/, accessed 2026-07-04.

[19] IEC. Industrial communication networks – network and system security. IEC 62443 series, 2021.

[20] H. Krawczyk and P. Eronen. Hmac-based extract-and-expand key derivation function (hkdf). IETF RFC 5869, 2010.

[21] MCUboot Project. Mcuboot secure bootloader. https://docs.mcuboot.com/, 2026. Accessed 2026-07-04.

[22] Microsoft Research and collaborators. Fido device onboard and riot-style derived device identity. Whitepapers and architecture notes, 2017.

[23] Charlie Miller and Chris Valasek. Remote exploitation of an unaltered passenger vehicle. In *Black Hat USA*, 2015.

[24] B. Moran, H. Tschofenig, et al. A firmware update architecture for internet of things devices. IETF RFC 9019, 2021.

[25] B. Moran, H. Tschofenig, et al. A concise binary object representation (cbor)-based serialization format for suit manifests. IETF RFC 9124, 2023.

[26] N. Moro, A. Dehbaoui, K. Heydemann, B. Robisson, and E. Encrenaz. Voltage glitching attacks on embedded systems. In *FDTC*, 2014.

[27] NIST. Secure hash standard (shs). FIPS PUB 180-4, 2015.

[28] NIST. Platform firmware resiliency guidelines. NIST SP 800-193, 2018.

[29] NIST. Recommendation for key management, part 1: General. NIST SP 800-57 Part 1 Rev. 5, 2020.

[30] NIST. Nist announces first four quantum-resistant cryptographic algorithms. NIST News Release, 2022.

[31] NIST. Secure software development framework (ssdf) version 1.1. NIST SP 800-218, 2022.

[32] NIST. Digital signature standard (dss). FIPS PUB 186-5, 2023.

[33] NIST. Module-lattice-based digital signature standard (ml-dsa). FIPS 204, 2024.

[34] NIST. Module-lattice-based key-encapsulation mechanism standard (ml-kem). FIPS 203, 2024.

[35] NIST. Stateless hash-based digital signature standard (slh-dsa). FIPS 205, 2024.

[36] NXP Semiconductors. Secure ota update on mcu platforms. Application Notes and Security Whitepapers, 2023.

[37] Timo Obermaier and Stefan Tatschner. Shedding too much light on a microcontroller's firmware protection. In *USENIX WOOT*, 2017.

[38] Colin O'Flynn et al. Security analysis of fault injection attacks against secure boot mechanisms. Selected embedded security conference papers, 2020.

[39] PSA Certified. Psa certified: Root of trust. https://www.psacertified.org/, 2024. Accessed 2026-07-04.

[40] RISC-V International. The risc-v instruction set manual, volume ii: Privileged architecture. Ratified specification, 2024.

[41] J. Schaad. Cbor object signing and encryption (cose): Structures and process. IETF RFC 9052, 2022.

[42] SLSA Contributors. Slsa v1.0 specification. https://slsa.dev/spec/v1.0/, 2023. Accessed 2026-07-04.

[43] STMicroelectronics. Eeprom emulation techniques and flash endurance on stm32. Application Note AN4894, 2023.

[44] The Update Framework Authors. The update framework (tuf) specification. https://theupdateframework.github.io/specification/latest/, 2024. Accessed 2026-07-04.

[45] Trusted Computing Group. Tcg dice attestation architecture. TCG Specification, 2018.

[46] TrustedFirmware.org. Trusted firmware-m documentation. https://trustedfirmware-m.readthedocs.io/, 2026. Accessed 2026-07-04.

[47] Uptane Community. Uptane standard for design and implementation. https://uptane.org/, 2024. Accessed 2026-07-04.

[48] USB Implementers Forum. Universal serial bus device class specification for device firmware upgrade, version 1.1. USB-IF specification, 2004. https://www.usb.org/sites/default/files/DFU_1.1.pdf, accessed 2026-07-04.

License